

Linux 与 ROS 2 嵌入式期末复习资料

复习讲义

2026 年 5 月 10 日

目录

1	Linux 基础概念	5
1.1	什么是 Linux	5
1.2	Linux 的核心思想	5
1.3	Linux 目录结构	6
2	Linux 常用命令	6
2.1	命令基本格式	6
2.2	获取帮助	7
2.3	目录操作	7
2.4	文件操作	8
3	文件查看与文本处理	9
3.1	grep 查找文本	9
3.2	find 查找文件	9
3.3	sed 文本替换	10
3.4	awk 文本处理	10
4	输入输出重定向与管道	11
4.1	标准输入、输出、错误	11
4.2	重定向	11
4.3	管道	11
5	用户、用户组与权限	12
5.1	用户相关命令	12
5.2	用户管理	12
5.3	文件权限	12
5.4	权限考试重点	13

目录	2
6 进程管理	14
6.1 查看进程	14
6.2 终止进程	14
6.3 前台与后台任务	15
7 软件包管理	15
7.1 Debian/Ubuntu: apt	15
7.2 RedHat/CentOS/Fedora: dnf 或 yum	16
8 压缩与归档	16
8.1 tar	16
8.2 zip 和 unzip	16
9 磁盘、文件系统与挂载	17
9.1 查看磁盘信息	17
9.2 挂载和卸载	17
10 网络基础命令	17
10.1 查看网络配置	17
10.2 测试网络	18
10.3 端口和连接	18
10.4 SSH 远程登录	18
11 环境变量	18
11.1 查看环境变量	18
11.2 设置环境变量	19
12 Vim 基础	19
12.1 Vim 三种常用模式	19
12.2 常用操作	19
13 Shell 脚本基础	20
13.1 第一个 Shell 脚本	20
13.2 变量	20
13.3 命令替换	21
13.4 位置参数	21
13.5 条件判断	21
13.6 case 分支	23
13.7 for 循环	23
13.8 while 循环	24
13.9 函数	24

13.10 退出状态码	24
13.11 实用脚本示例：自动备份目录	24
13.12 实用脚本示例：检查程序是否运行	25
14 编译、Makefile 与调试	26
14.1 gcc 编译 C 程序	26
14.2 Makefile 基础	27
14.3 gdb 调试	27
15 Linux 系统管理	28
15.1 systemd 服务管理	28
15.2 定时任务 cron	29
16 嵌入式 Linux 基础	29
16.1 嵌入式 Linux 启动流程	29
16.2 U-Boot 常用命令	30
16.3 内核启动参数	30
16.4 设备文件	30
16.5 proc 文件系统	31
16.6 sys 文件系统	31
16.7 设备树 Device Tree	32
16.8 内核模块	33
16.9 交叉编译	33
16.10 NFS 根文件系统	33
17 Linux C 编程基础	34
17.1 文件 I/O	34
17.2 进程创建 fork	35
17.3 线程 pthread	35
18 ROS 2 基础知识	36
18.1 ROS 2 是什么	36
18.2 ROS 2 核心概念	37
18.3 ROS 2 工作空间	37
18.4 ROS 2 常用命令	38
18.4.1 查看节点	38
18.4.2 查看话题	38
18.4.3 查看服务	38
18.4.4 查看参数	39
18.4.5 查看接口类型	39

目录	4
18.4.6 运行节点	39
18.5 创建 ROS 2 Python 功能包	39
18.6 Python 发布者示例	40
18.7 Python 订阅者示例	41
18.8 创建 ROS 2 C++ 功能包	42
18.9 C++ 发布者示例	42
18.10 CMakeLists.txt 示例	43
19 ROS 2 通信机制	44
19.1 Topic: 发布订阅	44
19.2 Service: 请求响应	44
19.3 Action: 长时间任务	45
19.4 Parameter: 参数	45
19.5 QoS 服务质量	45
20 ROS 2 Launch 文件	46
21 ROS 2 TF2 坐标变换基础	47
22 ROS 2 Bag 录包与回放	47
23 ROS 2 常见排错	48
23.1 找不到包	48
23.2 节点运行不了	48
23.3 话题没有数据	48
23.4 多机通信失败	49
24 Linux 与 ROS 2 期末速记	49
24.1 Linux 命令速记	49
24.2 ROS 2 命令速记	50
25 考试常考问答	51
25.1 Linux 为什么说一切皆文件?	51
25.2 绝对路径和相对路径区别是什么?	51
25.3 硬链接和软链接区别是什么?	51
25.4 进程和线程区别是什么?	51
25.5 ROS 2 Topic、Service、Action 区别是什么?	52
25.6 ROS 2 为什么不需要 ROS Master?	52
26 最后复习建议	52

1 Linux 基础概念

1.1 什么是 Linux

Linux 是一种类 Unix 操作系统内核, 常见发行版包括 Ubuntu、Debian、CentOS、Fedora、Arch Linux 等。

Linux 常用于:

- 服务器系统;
- 嵌入式系统;
- 机器人系统;
- 云计算平台;
- 开发环境。

Linux 系统一般由以下部分组成:

- Linux Kernel: 内核, 负责进程、内存、文件系统、设备驱动、网络等;
- Shell: 命令解释器, 例如 bash、zsh;
- GNU 工具: 如 ls、cp、gcc、make;
- 文件系统: 组织文件和设备;
- 用户空间程序: 用户运行的程序。

1.2 Linux 的核心思想

1. 一切皆文件。
2. 小工具组合完成复杂任务。
3. 使用文本文件作为配置文件。
4. 多用户、多任务。
5. 权限控制严格。

1.3 Linux 目录结构

目录	作用
/	根目录，所有目录的起点
/bin	基本命令，如 ls、cp、mv
/sbin	系统管理命令，如 ifconfig、reboot
/etc	配置文件目录
/home	普通用户家目录
/root	root 用户家目录
/usr	用户程序和库文件
/var	日志、缓存、可变数据
/tmp	临时文件
/dev	设备文件，如 /dev/ttyUSB0
/proc	虚拟文件系统，内核和进程信息
/sys	虚拟文件系统，设备和驱动信息
/boot	内核、启动文件
/lib	系统库文件
/mnt	临时挂载点
/media	自动挂载设备，如 U 盘

嵌入式中常见的重要目录：

- /dev：串口、I2C、SPI、GPIO 等设备文件；
- /proc：查看 CPU、内存、进程等信息；
- /sys：访问设备、驱动、GPIO、PWM 等接口；
- /etc/init.d 或 systemd：启动脚本和服务管理。

2 Linux 常用命令

2.1 命令基本格式

Linux 命令一般格式：

```
1 command [options] [arguments]
```

例如：

```
1 ls -l /home
```

含义：

- `ls`: 命令;
- `-l`: 选项, 表示长格式显示;
- `/home`: 参数, 表示查看 `/home` 目录。

2.2 获取帮助

```
1 man ls
2 ls --help
3 info ls
```

例如查看 `cp` 命令帮助:

```
1 man cp
2 cp --help
```

2.3 目录操作

命令	作用
<code>pwd</code>	显示当前所在目录
<code>cd</code>	切换目录
<code>ls</code>	查看目录内容
<code>mkdir</code>	创建目录
<code>rmdir</code>	删除空目录
<code>tree</code>	以树形显示目录结构

示例:

```
1 pwd
2 cd /home
3 cd ..
4 cd ~
5 mkdir test
6 mkdir -p dir1/dir2/dir3
7 rmdir test
8 ls
9 ls -l
10 ls -a
11 ls -lh
```

解释:

- `cd ..`: 返回上一级目录;
- `cd ~`: 进入当前用户家目录;
- `mkdir -p`: 递归创建多级目录;
- `ls -a`: 显示隐藏文件;
- `ls -lh`: 以人类可读方式显示文件大小。

2.4 文件操作

命令	作用
<code>touch</code>	创建空文件或更新时间戳
<code>cp</code>	复制文件或目录
<code>mv</code>	移动或重命名文件
<code>rm</code>	删除文件或目录
<code>cat</code>	查看文件内容
<code>less</code>	分页查看文件
<code>head</code>	查看文件开头
<code>tail</code>	查看文件结尾
<code>file</code>	查看文件类型
<code>stat</code>	查看文件详细属性

示例:

```
1 touch a.txt
2 cp a.txt b.txt
3 cp -r dir1 dir2
4 mv b.txt c.txt
5 rm c.txt
6 rm -r dir2
7 rm -rf dir2
8 cat a.txt
9 less a.txt
10 head -n 5 a.txt
11 tail -n 10 a.txt
12 tail -f /var/log/syslog
13 file a.txt
14 stat a.txt
```

注意:

- `rm -rf` 非常危险，表示强制递归删除；
- `tail -f` 常用于实时查看日志；
- 嵌入式调试时常用 `dmesg` 和 `tail -f` 查看驱动或系统日志。

3 文件查看与文本处理

3.1 grep 查找文本

`grep` 用于在文件中查找匹配内容。

```
1 grep "main" test.c
2 grep -n "error" log.txt
3 grep -i "warning" log.txt
4 grep -r "TODO" ./src
5 grep -v "debug" log.txt
```

常用选项：

- `-n`：显示行号；
- `-i`：忽略大小写；
- `-r`：递归查找；
- `-v`：反向匹配。

例子：查找系统日志中的错误：

```
1 grep -i "error" /var/log/syslog
```

3.2 find 查找文件

```
1 find /home -name "*.c"
2 find . -type f -name "*.txt"
3 find . -type d -name "build"
4 find . -size +10M
5 find . -mtime -7
```

含义：

- `-name`：按文件名查找；
- `-type f`：普通文件；

- `-type d`: 目录;
- `-size +10M`: 大于 10 MB 的文件;
- `-mtime -7`: 7 天内修改过的文件。

删除所有 build 目录:

```
1 find . -type d -name "build" -exec rm -rf {} \;
```

3.3 sed 文本替换

`sed` 常用于按行处理文本。

```
1 sed 's/old/new/' file.txt
2 sed 's/old/new/g' file.txt
3 sed -i 's/old/new/g' file.txt
```

解释:

- `s/old/new/`: 替换每行第一个 old;
- `s/old/new/g`: 替换所有 old;
- `-i`: 直接修改原文件。

示例: 把配置文件中的 115200 改成 9600:

```
1 sed -i 's/115200/9600/g' uart.conf
```

3.4 awk 文本处理

`awk` 适合处理列数据。

```
1 awk '{print $1}' file.txt
2 awk '{print $1, $3}' file.txt
3 awk -F ':' '{print $1}' /etc/passwd
```

示例: 查看当前系统用户名:

```
1 awk -F ':' '{print $1}' /etc/passwd
```

示例: 查看磁盘使用率:

```
1 df -h | awk '{print $1, $5, $6}'
```

4 输入输出重定向与管道

4.1 标准输入、输出、错误

Linux 中有三个标准文件描述符：

名称	编号	含义
stdin	0	标准输入
stdout	1	标准输出
stderr	2	标准错误

4.2 重定向

```
1 echo "hello" > a.txt
2 echo "world" >> a.txt
3 cat < a.txt
4 ls not_exist 2> error.log
5 ls /home > out.log 2> err.log
6 ls /home > all.log 2>&1
```

解释：

- >：覆盖写入；
- >>：追加写入；
- 2>：重定向错误输出；
- 2>&1：把错误输出合并到标准输出。

4.3 管道

管道 | 用于把前一个命令的输出作为后一个命令的输入。

```
1 ps aux | grep ssh
2 dmesg | grep tty
3 cat file.txt | grep "error" | wc -l
```

示例：统计日志中 error 出现次数：

```
1 grep -i "error" system.log | wc -l
```

5 用户、用户组与权限

5.1 用户相关命令

```
1 whoami
2 id
3 who
4 W
5 su
6 sudo command
7 passwd
```

5.2 用户管理

```
1 sudo useradd testuser
2 sudo passwd testuser
3 sudo userdel testuser
4 sudo userdel -r testuser
5 sudo groupadd dev
6 sudo usermod -aG dev testuser
```

解释：

- useradd: 添加用户；
- passwd: 设置密码；
- userdel -r: 删除用户及其家目录；
- usermod -aG: 把用户加入组。

5.3 文件权限

查看文件权限：

```
1 ls -l
```

输出示例：

```
1 -rwxr-xr-- 1 user group 1024 May 10 test.sh
```

含义：

- 第 1 位：文件类型，- 表示普通文件，d 表示目录；

- 后 9 位：权限；
- 每 3 位一组：所有者、所属组、其他用户；
- r：读，w：写，x：执行。

权限数字表示：

权限	数字
r	4
w	2
x	1
rwX	7
rw-	6
r-X	5
r-	4

修改权限：

```
1 chmod 755 test.sh
2 chmod +x test.sh
3 chmod u+x test.sh
4 chmod g-w test.sh
5 chmod o-r test.sh
```

修改所有者和组：

```
1 sudo chown user file.txt
2 sudo chown user:group file.txt
3 sudo chgrp group file.txt
```

5.4 权限考试重点

```
1 chmod 755 file
```

表示：

- 所有者： $7 = r + w + x$ ；
- 所属组： $5 = r + x$ ；
- 其他用户： $5 = r + x$ 。

```
1 chmod 644 file
```

表示：

- 所有者可读写；
- 组用户只读；
- 其他用户只读。

6 进程管理

6.1 查看进程

```
1 ps
2 ps aux
3 top
4 htop
5 pidof sshd
6 pgrep ssh
```

常见命令：

```
1 ps aux | grep nginx
```

6.2 终止进程

```
1 kill PID
2 kill -9 PID
3 pkill process_name
4 killall process_name
```

常用信号：

信号	数字	作用
SIGTERM	15	正常终止进程
SIGKILL	9	强制杀死进程
SIGINT	2	Ctrl + C 产生，中断进程
SIGHUP	1	挂起，常用于重新加载配置

示例：

```
1 ps aux | grep my_app
2 kill 1234
3 kill -9 1234
```

6.3 前台与后台任务

```
1 ./program
2 ./program &
3 jobs
4 fg %1
5 bg %1
6 Ctrl + Z
7 Ctrl + C
```

解释：

- &: 后台运行；
- jobs: 查看后台任务；
- fg: 切回前台；
- bg: 后台继续运行；
- Ctrl + Z: 暂停任务；
- Ctrl + C: 终止任务。

7 软件包管理

7.1 Debian/Ubuntu: apt

```
1 sudo apt update
2 sudo apt upgrade
3 sudo apt install gcc make git
4 sudo apt remove package_name
5 sudo apt purge package_name
6 sudo apt search package_name
7 sudo apt show package_name
```

示例：安装 C 语言开发工具：

```
1 sudo apt update
2 sudo apt install build-essential
```

7.2 RedHat/CentOS/Fedora: dnf 或 yum

```
1 sudo dnf install gcc make git
2 sudo dnf remove package_name
3 sudo dnf search package_name
```

较老系统可能使用:

```
1 sudo yum install gcc make git
```

8 压缩与归档

8.1 tar

```
1 tar -cvf archive.tar dir
2 tar -xvf archive.tar
3 tar -czvf archive.tar.gz dir
4 tar -xzvf archive.tar.gz
5 tar -cjvf archive.tar.bz2 dir
6 tar -xjvf archive.tar.bz2
```

选项含义:

- c: 创建;
- x: 解压;
- v: 显示过程;
- f: 指定文件;
- z: gzip;
- j: bzip2。

示例: 打包工程目录:

```
1 tar -czvf project_backup.tar.gz project/
```

8.2 zip 和 unzip

```
1 zip file.zip a.txt b.txt
2 zip -r project.zip project/
3 unzip project.zip
```


9 磁盘、文件系统与挂载

9.1 查看磁盘信息

```
1 df -h
2 du -sh *
3 lsblk
4 fdisk -l
5 mount
```

解释：

- `df -h`: 查看文件系统空间；
- `du -sh`: 查看文件或目录大小；
- `lsblk`: 查看块设备；
- `fdisk -l`: 查看磁盘分区；
- `mount`: 查看挂载信息。

9.2 挂载和卸载

```
1 sudo mount /dev/sdb1 /mnt
2 sudo umount /mnt
```

嵌入式中常见挂载：

```
1 mount -t nfs 192.168.1.100:/home/nfs /mnt
```

表示通过 NFS 挂载主机目录到开发板。

10 网络基础命令

10.1 查看网络配置

```
1 ip addr
2 ip link
3 ifconfig
4 hostname
5 hostname -I
```

推荐新命令：

```
1 ip addr show
2 ip route show
```

10.2 测试网络

```
1 ping 8.8.8.8
2 ping www.baidu.com
3 traceroute www.baidu.com
4 curl www.example.com
5 wget http://example.com/file.tar.gz
```

10.3 端口和连接

```
1 netstat -tulnp
2 ss -tulnp
3 lsof -i :8080
```

示例：查看 8080 端口被谁占用：

```
1 sudo lsof -i :8080
```

10.4 SSH 远程登录

```
1 ssh user@192.168.1.10
2 scp file.txt user@192.168.1.10:/home/user/
3 scp user@192.168.1.10:/home/user/a.txt .
```

嵌入式开发中常用 SSH 登录开发板：

```
1 ssh root@192.168.1.100
```

11 环境变量

11.1 查看环境变量

```
1 env
2 printenv
3 echo $PATH
4 echo $HOME
5 echo $USER
```

11.2 设置环境变量

临时设置：

```
1 export MYVAR=hello
2 echo $MYVAR
```

永久设置可写入：

```
1 ~/.bashrc
2 ~/.profile
3 /etc/profile
```

示例：

```
1 echo 'export PATH=$PATH:/opt/toolchain/bin' >> ~/.bashrc
2 source ~/.bashrc
```

嵌入式交叉编译经常设置工具链路径：

```
1 export PATH=$PATH:/opt/arm-linux-gcc/bin
2 export CROSS_COMPILE=arm-linux-gnueabi-
3 export ARCH=arm
```

12 Vim 基础

12.1 Vim 三种常用模式

- 普通模式：移动光标、复制、删除；
- 插入模式：输入文本；
- 命令模式：保存、退出、查找。

12.2 常用操作

命令	作用
i	进入插入模式
Esc	返回普通模式
:w	保存
:q	退出
:wq	保存并退出
:q!	强制退出不保存

dd	删除一行
yy	复制一行
p	粘贴
/word	查找 word
n	下一个匹配
u	撤销

13 Shell 脚本基础

13.1 第一个 Shell 脚本

创建 hello.sh:

```
1 #!/bin/bash
2
3 echo "Hello Linux"
```

执行:

```
1 chmod +x hello.sh
2 ./hello.sh
```

解释:

- #!/bin/bash: 指定脚本解释器;
- chmod +x: 添加可执行权限。

13.2 变量

```
1 #!/bin/bash
2
3 name="Linux"
4 echo "Hello $name"
5 echo "Hello ${name}"
```

注意:

- 变量赋值时等号两边不能有空格;
- 使用变量时加 \$。

13.3 命令替换

```
1 #!/bin/bash
2
3 now=$(date)
4 echo "Current time: $now"
5
6 files=$(ls)
7 echo "$files"
```

13.4 位置参数

```
1 #!/bin/bash
2
3 echo "Script name: $0"
4 echo "First arg: $1"
5 echo "Second arg: $2"
6 echo "All args: $@"
7 echo "Arg count: $#"
```

执行:

```
1 ./test.sh hello world
```

输出中:

- \$0: 脚本名;
- \$1: 第一个参数;
- \$2: 第二个参数;
- \$@: 所有参数;
- \$#: 参数个数。

13.5 条件判断

```
1 #!/bin/bash
2
3 num=10
4
5 if [ $num -gt 5 ]; then
6     echo "num > 5"
```

```
7 else
8     echo "num <= 5"
9 fi
```

常用数字比较：

表达式	含义
-eq	等于
-ne	不等于
-gt	大于
-lt	小于
-ge	大于等于
-le	小于等于

字符串比较：

```
1 #!/bin/bash
2
3 str="yes"
4
5 if [ "$str" = "yes" ]; then
6     echo "OK"
7 fi
```

文件判断：

表达式	含义
-e file	文件存在
-f file	普通文件
-d dir	目录
-r file	可读
-w file	可写
-x file	可执行

示例：

```
1 #!/bin/bash
2
3 if [ -f "main.c" ]; then
4     echo "main.c exists"
5 else
6     echo "main.c not found"
```

```
7 fi
```

13.6 case 分支

```
1 #!/bin/bash
2
3 case "$1" in
4     start)
5         echo "start service"
6         ;;
7     stop)
8         echo "stop service"
9         ;;
10    restart)
11        echo "restart service"
12        ;;
13    *)
14        echo "Usage: $0 {start|stop|restart}"
15        ;;
16 esac
```

13.7 for 循环

```
1 #!/bin/bash
2
3 for file in *.c
4 do
5     echo "C file: $file"
6 done
```

数字循环:

```
1 #!/bin/bash
2
3 for i in {1..5}
4 do
5     echo $i
6 done
```

13.8 while 循环

```
1 #!/bin/bash
2
3 count=1
4
5 while [ $count -le 5 ]
6 do
7     echo $count
8     count=$((count + 1))
9 done
```

13.9 函数

```
1 #!/bin/bash
2
3 say_hello() {
4     echo "Hello $1"
5 }
6
7 say_hello Linux
8 say_hello ROS2
```

13.10 退出状态码

每条命令执行后都有返回值，\$? 表示上一条命令状态：

```
1 ls /home
2 echo $?
3
4 ls /not_exist
5 echo $?
```

一般：

- 0 表示成功；
- 非 0 表示失败。

13.11 实用脚本示例：自动备份目录


```
1 #!/bin/bash
2
3 src_dir="$1"
4 backup_dir="$2"
5
6 if [ $# -ne 2 ]; then
7     echo "Usage: $0 src_dir backup_dir"
8     exit 1
9 fi
10
11 if [ ! -d "$src_dir" ]; then
12     echo "Source directory not found"
13     exit 2
14 fi
15
16 mkdir -p "$backup_dir"
17
18 time_str=$(date +%Y%m%d_%H%M%S)
19 backup_file="$backup_dir/backup_$time_str.tar.gz"
20
21 tar -czvf "$backup_file" "$src_dir"
22
23 echo "Backup saved to $backup_file"
```

执行:

```
1 chmod +x backup.sh
2 ./backup.sh project backups
```

13.12 实用脚本示例：检查程序是否运行

```
1 #!/bin/bash
2
3 process_name="$1"
4
5 if [ $# -ne 1 ]; then
6     echo "Usage: $0 process_name"
7     exit 1
8 fi
9
10 pid=$(pgrep "$process_name")
```

```
11
12 if [ -n "$pid" ]; then
13     echo "$process_name is running, pid=$pid"
14 else
15     echo "$process_name is not running"
16 fi
```

14 编译、Makefile 与调试

14.1 gcc 编译 C 程序

假设有 hello.c:

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello Linux\n");
6     return 0;
7 }
```

编译:

```
1 gcc hello.c -o hello
2 ./hello
```

带警告和调试信息:

```
1 gcc -Wall -g hello.c -o hello
```

常用选项:

- -o: 指定输出文件;
- -Wall: 打开常见警告;
- -g: 生成调试信息;
- -I: 指定头文件目录;
- -L: 指定库目录;
- -l: 链接库。

示例:

```
1 gcc main.c -I./include -L./lib -lmylib -o app
```

14.2 Makefile 基础

简单 Makefile:

```
1 app: main.o add.o
2     gcc main.o add.o -o app
3
4 main.o: main.c
5     gcc -c main.c
6
7 add.o: add.c
8     gcc -c add.c
9
10 clean:
11     rm -f *.o app
```

注意：命令前面必须是 Tab，不是空格。

执行：

```
1 make
2 make clean
```

更常见写法：

```
1 CC = gcc
2 CFLAGS = -Wall -g
3 TARGET = app
4 OBJS = main.o add.o
5
6 $(TARGET): $(OBJS)
7     $(CC) $(OBJS) -o $(TARGET)
8
9 %.o: %.c
10     $(CC) $(CFLAGS) -c $< -o $@
11
12 clean:
13     rm -f $(OBJS) $(TARGET)
```

14.3 gdb 调试

编译时添加 -g:

```
1 gcc -g main.c -o app
2 gdb ./app
```

gdb 常用命令:

命令	作用
break main	在 main 函数设置断点
b 10	在第 10 行设置断点
run	运行程序
next	单步执行, 不进入函数
step	单步执行, 进入函数
continue	继续运行
print var	打印变量
backtrace	查看函数调用栈
quit	退出 gdb

示例:

```
1 (gdb) break main
2 (gdb) run
3 (gdb) next
4 (gdb) print i
5 (gdb) continue
6 (gdb) quit
```

15 Linux 系统管理

15.1 systemd 服务管理

```
1 systemctl status ssh
2 sudo systemctl start ssh
3 sudo systemctl stop ssh
4 sudo systemctl restart ssh
5 sudo systemctl enable ssh
6 sudo systemctl disable ssh
```

查看日志:

```
1 journalctl
2 journalctl -u ssh
3 journalctl -f
4 journalctl -u ssh -f
```

15.2 定时任务 cron

编辑当前用户定时任务：

```
1 crontab -e
```

查看：

```
1 crontab -l
```

格式：

```
1 分钟 小时 日期 月份 星期 命令
```

例子：每天凌晨 2 点执行备份：

```
1 0 2 * * * /home/user/backup.sh
```

每 5 分钟执行一次：

```
1 */5 * * * * /home/user/check.sh
```

16 嵌入式 Linux 基础

16.1 嵌入式 Linux 启动流程

典型启动流程：

1. 上电复位；
2. BootROM 执行；
3. 加载 Bootloader，例如 U-Boot；
4. Bootloader 初始化 DDR、Flash、串口等硬件；
5. Bootloader 加载 Linux Kernel；
6. Kernel 解压并启动；
7. Kernel 挂载根文件系统 rootfs；
8. 启动第一个用户空间进程 init 或 systemd；
9. 启动各种服务和应用程序。

简化流程：

```
1 BootROM -> U-Boot -> Linux Kernel -> RootFS -> init/systemd -> App
```

16.2 U-Boot 常用命令

```
1 printenv
2 setenv bootargs console=ttyS0,115200 root=/dev/mmcblk0p2 rw
3 saveenv
4 boot
5 bootm
6 tftpboot
7 mmc list
8 mmc dev 0
9 fatls mmc 0:1
10 fatload mmc 0:1 0x80000000 zImage
```

常见环境变量：

- bootargs: 传给 Linux 内核的启动参数；
- bootcmd: 自动启动命令；
- ethaddr: 网卡 MAC 地址；
- ipaddr: 开发板 IP；
- serverip: 服务器 IP。

16.3 内核启动参数

示例：

```
1 console=ttyS0,115200 root=/dev/mmcblk0p2 rw rootwait
```

含义：

- console=ttyS0,115200: 串口控制台；
- root=/dev/mmcblk0p2: 根文件系统位置；
- rw: 以读写方式挂载；
- rootwait: 等待根设备准备好。

16.4 设备文件

Linux 中设备也表现为文件，通常在 /dev 目录下。

常见设备：

设备文件	含义
/dev/ttyS0	串口设备
/dev/ttyUSB0	USB 转串口设备
/dev/mmcblk0	eMMC 或 SD 卡
/dev/sda	SATA 或 USB 存储设备
/dev/i2c-0	I2C 总线设备
/dev/spidev0.0	SPI 设备
/dev/fb0	Framebuffer 显示设备
/dev/input/event0	输入设备

查看串口设备：

```
1 ls /dev/ttyS*
2 ls /dev/ttyUSB*
3 dmesg | grep tty
```

16.5 proc 文件系统

/proc 是虚拟文件系统，用于查看内核和进程信息。

```
1 cat /proc/cpuinfo
2 cat /proc/meminfo
3 cat /proc/version
4 cat /proc/cmdline
5 cat /proc/interrupts
6 cat /proc/filesystems
```

嵌入式常用：

```
1 cat /proc/cmdline
2 cat /proc/interrupts
3 cat /proc/meminfo
```

16.6 sys 文件系统

/sys 用于表示设备、驱动、总线等内核对象。

常见路径：

```
1 /sys/class
2 /sys/bus
3 /sys/devices
```

```
4 /sys/class/gpio
5 /sys/class/leds
6 /sys/class/pwm
```

GPIO 旧式 sysfs 操作示例:

```
1 echo 17 > /sys/class/gpio/export
2 echo out > /sys/class/gpio/gpio17/direction
3 echo 1 > /sys/class/gpio/gpio17/value
4 echo 0 > /sys/class/gpio/gpio17/value
5 echo 17 > /sys/class/gpio/unexport
```

注意: 较新的 Linux 内核更推荐使用 libgpiod。

16.7 设备树 Device Tree

设备树用于描述硬件信息, 让内核知道板级硬件资源。

常见文件:

- .dts: 设备树源文件;
- .dtsi: 设备树包含文件;
- .dtb: 编译后的设备树二进制文件。

设备树示例:

```
1 uart0: serial@101f1000 {
2     compatible = "arm,pl011";
3     reg = <0x101f1000 0x1000>;
4     interrupts = <5>;
5     status = "okay";
6 };
```

常见属性:

- compatible: 用于驱动匹配;
- reg: 寄存器地址和大小;
- interrupts: 中断号;
- status: 设备是否启用。

16.8 内核模块

查看模块：

```
1 lsmod
```

加载模块：

```
1 sudo insmod hello.ko
2 sudo modprobe driver_name
```

卸载模块：

```
1 sudo rmmod hello
```

查看内核日志：

```
1 dmesg
2 dmesg | tail
3 dmesg -w
```

16.9 交叉编译

交叉编译是指在一种平台上编译另一种平台运行的程序。

例如在 x86 PC 上编译 ARM 程序：

```
1 arm-linux-gnueabi-gcc main.c -o app
```

常见环境变量：

```
1 export ARCH=arm
2 export CROSS_COMPILE=arm-linux-gnueabi-
```

编译内核示例：

```
1 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- menuconfig
2 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- zImage
3 make ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- dtbs
```

16.10 NFS 根文件系统

开发阶段常用 NFS 挂载根文件系统，方便调试。

主机端安装 NFS：

```
1 sudo apt install nfs-kernel-server
```

配置 /etc/exports：

```
1 /home/user/nfsroot *(rw,sync,no_root_squash,no_subtree_check)
```

重启服务:

```
1 sudo exportfs -ra
2 sudo systemctl restart nfs-kernel-server
```

开发板挂载:

```
1 mount -t nfs 192.168.1.10:/home/user/nfsroot /mnt
```

17 Linux C 编程基础

17.1 文件 I/O

系统调用方式:

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     int fd;
8     char buf[100];
9
10    fd = open("test.txt", O_RDONLY);
11    if (fd < 0) {
12        perror("open");
13        return 1;
14    }
15
16    int n = read(fd, buf, sizeof(buf) - 1);
17    if (n < 0) {
18        perror("read");
19        close(fd);
20        return 1;
21    }
22
23    buf[n] = '\0';
24    printf("%s\n", buf);
25
26    close(fd);
27    return 0;
28 }
```

常见系统调用：

- open: 打开文件;
- read: 读取文件;
- write: 写文件;
- close: 关闭文件;
- ioctl: 设备控制，驱动开发常见。

17.2 进程创建 fork

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     pid_t pid = fork();
7
8     if (pid < 0) {
9         perror("fork");
10        return 1;
11    } else if (pid == 0) {
12        printf("Child process\n");
13    } else {
14        printf("Parent process, child pid=%d\n", pid);
15    }
16
17    return 0;
18 }
```

17.3 线程 pthread

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void *thread_func(void *arg)
5 {
6     printf("Hello from thread\n");
```

```
7   return NULL;
8 }
9
10 int main(void)
11 {
12     pthread_t tid;
13
14     pthread_create(&tid, NULL, thread_func, NULL);
15     pthread_join(tid, NULL);
16
17     return 0;
18 }
```

编译:

```
1 gcc thread.c -o thread -lpthread
```

18 ROS 2 基础知识

18.1 ROS 2 是什么

ROS 2 全称 Robot Operating System 2, 不是传统意义上的操作系统, 而是机器人软件开发框架。

ROS 2 提供:

- 节点通信;
- 话题发布订阅;
- 服务请求响应;
- 动作通信;
- 参数管理;
- 坐标变换;
- 日志系统;
- 包管理和构建工具。

ROS 2 与 ROS 1 的重要区别:

- ROS 2 基于 DDS 通信;

- 支持实时性更好；
- 支持多机器人通信；
- 支持 QoS；
- 支持生命周期节点；
- 不再依赖 ROS Master。

18.2 ROS 2 核心概念

概念	含义
Node	节点，一个独立功能模块
Topic	话题，用于发布订阅通信
Publisher	发布者，向 Topic 发送消息
Subscriber	订阅者，从 Topic 接收消息
Service	服务，请求-响应模式
Client	服务客户端，发送请求
Server	服务端，处理请求
Action	动作，适合长时间任务
Message	消息类型
Parameter	参数，节点运行时配置
Launch	启动文件，用于同时启动多个节点
Workspace	工作空间，存放 ROS 2 工程
Package	功能包，ROS 2 基本组织单位
QoS	服务质量策略
DDS	ROS 2 底层通信机制

18.3 ROS 2 工作空间

典型工作空间结构：

```

1 ros2_ws/
2   src/
3     my_package/
4   build/
5   install/
6   log/

```

创建工作空间：

```
1 mkdir -p ~/ros2_ws/src
2 cd ~/ros2_ws
3 colcon build
4 source install/setup.bash
```

每次打开新终端后需要：

```
1 source /opt/ros/humble/setup.bash
2 source ~/ros2_ws/install/setup.bash
```

也可以写入 ~/.bashrc：

```
1 echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
2 echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
3 source ~/.bashrc
```

18.4 ROS 2 常用命令

18.4.1 查看节点

```
1 ros2 node list
2 ros2 node info /node_name
```

18.4.2 查看话题

```
1 ros2 topic list
2 ros2 topic info /topic_name
3 ros2 topic echo /topic_name
4 ros2 topic hz /topic_name
5 ros2 topic pub /chatter std_msgs/msg/String "{data: 'hello'}"
```

示例：

```
1 ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
2 "{linear: {x: 0.2, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.5}}"
```

18.4.3 查看服务

```
1 ros2 service list
2 ros2 service type /service_name
3 ros2 service call /service_name service_type "{request}"
```

示例：

```
1 ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 1, b: 2}"
```

18.4.4 查看参数

```
1 ros2 param list
2 ros2 param get /node_name parameter_name
3 ros2 param set /node_name parameter_name value
4 ros2 param dump /node_name
```

18.4.5 查看接口类型

```
1 ros2 interface list
2 ros2 interface show std_msgs/msg/String
3 ros2 interface show geometry_msgs/msg/Twist
```

18.4.6 运行节点

```
1 ros2 run package_name executable_name
```

示例：

```
1 ros2 run turtlesim turtlesim_node
2 ros2 run turtlesim turtle_teleop_key
```

18.5 创建 ROS 2 Python 功能包

创建包：

```
1 cd ~/ros2_ws/src
2 ros2 pkg create my_py_pkg --build-type ament_python --dependencies rclpy std_msgs
```

目录结构大致为：

```
1 my_py_pkg/
2   my_py_pkg/
3     __init__.py
4   package.xml
5   setup.py
6   setup.cfg
```

18.6 Python 发布者示例

文件: my_py_pkg/talker.py

```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5 class Talker(Node):
6     def __init__(self):
7         super().__init__('talker')
8         self.publisher = self.create_publisher(String, 'chatter', 10)
9         self.timer = self.create_timer(1.0, self.timer_callback)
10        self.count = 0
11
12    def timer_callback(self):
13        msg = String()
14        msg.data = f'Hello ROS 2: {self.count}'
15        self.publisher.publish(msg)
16        self.get_logger().info(f'Publish: {msg.data}')
17        self.count += 1
18
19 def main(args=None):
20     rclpy.init(args=args)
21     node = Talker()
22     rclpy.spin(node)
23     node.destroy_node()
24     rclpy.shutdown()
25
26 if __name__ == '__main__':
27     main()
```

在 setup.py 中添加入口:

```
1 entry_points={
2     'console_scripts': [
3         'talker = my_py_pkg.talker:main',
4     ],
5 }
```

编译运行:

```
1 cd ~/ros2_ws
2 colcon build
```



```
3 source install/setup.bash
4 ros2 run my_py_pkg talker
```

18.7 Python 订阅者示例

文件: my_py_pkg/listener.py

```
1 import rclpy
2 from rclpy.node import Node
3 from std_msgs.msg import String
4
5 class Listener(Node):
6     def __init__(self):
7         super().__init__('listener')
8         self.subscription = self.create_subscription(
9             String,
10            'chatter',
11            self.callback,
12            10
13        )
14
15     def callback(self, msg):
16         self.get_logger().info(f'I heard: {msg.data}')
17
18 def main(args=None):
19     rclpy.init(args=args)
20     node = Listener()
21     rclpy.spin(node)
22     node.destroy_node()
23     rclpy.shutdown()
24
25 if __name__ == '__main__':
26     main()
```

在 setup.py 中添加:

```
1 entry_points={
2     'console_scripts': [
3         'talker = my_py_pkg.talker:main',
4         'listener = my_py_pkg.listener:main',
5     ],
6 }
```

编译运行:

```
1 cd ~/ros2_ws
2 colcon build
3 source install/setup.bash
4
5 ros2 run my_py_pkg talker
6 ros2 run my_py_pkg listener
```

18.8 创建 ROS 2 C++ 功能包

```
1 cd ~/ros2_ws/src
2 ros2 pkg create my_cpp_pkg --build-type ament_cmake --dependencies rclcpp std_msgs
```

18.9 C++ 发布者示例

文件: src/talker.cpp

```
1 #include <chrono>
2 #include <memory>
3 #include <string>
4
5 #include "rclcpp/rclcpp.hpp"
6 #include "std_msgs/msg/string.hpp"
7
8 using namespace std::chrono_literals;
9
10 class Talker : public rclcpp::Node
11 {
12 public:
13     Talker() : Node("cpp_talker"), count_(0)
14     {
15         publisher_ = this->create_publisher<std_msgs::msg::String>("chatter", 10);
16
17         timer_ = this->create_wall_timer(
18             1s,
19             std::bind(&Talker::timer_callback, this)
20         );
21     }
22
23 private:
```

```

24 void timer_callback()
25 {
26     auto msg = std_msgs::msg::String();
27     msg.data = "Hello ROS 2 C++: " + std::to_string(count_++);
28     publisher_->publish(msg);
29     RCLCPP_INFO(this->get_logger(), "Publish: '%s'", msg.data.c_str());
30 }
31
32 rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
33 rclcpp::TimerBase::SharedPtr timer_;
34 size_t count_;
35 };
36
37 int main(int argc, char * argv[])
38 {
39     rclcpp::init(argc, argv);
40     rclcpp::spin(std::make_shared<Talker>());
41     rclcpp::shutdown();
42     return 0;
43 }

```

18.10 CMakeLists.txt 示例

```

1 cmake_minimum_required(VERSION 3.8)
2 project(my_cpp_pkg)
3
4 find_package(ament_cmake REQUIRED)
5 find_package(rclcpp REQUIRED)
6 find_package(std_msgs REQUIRED)
7
8 add_executable(talker src/talker.cpp)
9 ament_target_dependencies(talker rclcpp std_msgs)
10
11 install(TARGETS
12     talker
13     DESTINATION lib/${PROJECT_NAME}
14 )
15
16 ament_package()

```

编译运行：

```
1 cd ~/ros2_ws
2 colcon build
3 source install/setup.bash
4 ros2 run my_cpp_pkg talker
```

19 ROS 2 通信机制

19.1 Topic：发布订阅

Topic 适合连续数据流，例如：

- 摄像头图像；
- 激光雷达数据；
- 速度控制命令；
- 机器人位姿；
- IMU 数据。

特点：

- 异步通信；
- 多个发布者和多个订阅者；
- 不要求发送方等待接收方回复。

19.2 Service：请求响应

Service 适合短时间请求，例如：

- 查询状态；
- 开关设备；
- 重置机器人；
- 请求一次计算。

特点：

- 同步或异步请求响应；
- 客户端发送请求；
- 服务端返回结果。

19.3 Action：长时间任务

Action 适合耗时任务，例如：

- 导航到目标点；
- 机械臂执行轨迹；
- 自动充电；
- 巡航任务。

Action 包含：

- Goal：目标；
- Feedback：执行过程反馈；
- Result：最终结果。

19.4 Parameter：参数

参数用于配置节点。

命令示例：

```
1 ros2 param list
2 ros2 param get /my_node speed
3 ros2 param set /my_node speed 1.0
```

Python 节点中声明参数：

```
1 self.declare_parameter('speed', 1.0)
2 speed = self.get_parameter('speed').value
```

19.5 QoS 服务质量

ROS 2 QoS 用于控制通信质量。

常见策略：

- Reliability：可靠性；
- Durability：持久性；
- History：历史缓存；
- Depth：队列深度；
- Deadline：截止时间；

- Lifespan: 消息寿命。

常见可靠性:

- Reliable: 可靠传输, 适合重要数据;
- Best Effort: 尽力传输, 适合传感器高速数据。

示例:

- 激光雷达、摄像头: 常用 Best Effort;
- 控制命令、状态: 常用 Reliable。

20 ROS 2 Launch 文件

Launch 文件用于一次启动多个节点。

Python launch 示例:

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 def generate_launch_description():
5     return LaunchDescription([
6         Node(
7             package='my_py_pkg',
8             executable='talker',
9             name='talker_node'
10        ),
11        Node(
12            package='my_py_pkg',
13            executable='listener',
14            name='listener_node'
15        )
16    ])
```

运行:

```
1 ros2 launch my_py_pkg demo.launch.py
```

安装 launch 文件时, 在 `setup.py` 中通常需要配置数据文件。

21 ROS 2 TF2 坐标变换基础

TF2 用于维护机器人各坐标系之间的关系。

常见坐标系：

- map：地图坐标系；
- odom：里程计坐标系；
- base_link：机器人本体坐标系；
- laser：雷达坐标系；
- camera_link：相机坐标系。

常见关系：

```
1 map -> odom -> base_link -> laser
2           |
3           -> camera_link
```

查看 TF：

```
1 ros2 run tf2_tools view_frames
2 ros2 run tf2_ros tf2_echo map base_link
```

发布静态 TF：

```
1 ros2 run tf2_ros static_transform_publisher \
2 0 0 0 0 0 0 base_link laser
```

22 ROS 2 Bag 录包与回放

录制所有话题：

```
1 ros2 bag record -a
```

录制指定话题：

```
1 ros2 bag record /chatter /cmd_vel
```

查看 bag 信息：

```
1 ros2 bag info bag_directory
```

回放：

```
1 ros2 bag play bag_directory
```

常用于：

- 记录传感器数据；
- 离线调试算法；
- 比赛和实验数据复现。

23 ROS 2 常见排错

23.1 找不到包

现象：

```
1 Package 'xxx' not found
```

解决：

```
1 cd ~/ros2_ws
2 colcon build
3 source install/setup.bash
4 ros2 pkg list | grep xxx
```

23.2 节点运行不了

检查：

```
1 ros2 pkg executables package_name
2 ros2 run package_name executable_name
```

Python 包还要检查 `setup.py` 中的 `entry_points`。

23.3 话题没有数据

检查：

```
1 ros2 topic list
2 ros2 topic info /topic_name
3 ros2 topic echo /topic_name
4 ros2 node list
5 ros2 node info /node_name
```

可能原因：

- 节点没启动；

- 话题名不一致；
- 消息类型不一致；
- QoS 不匹配；
- 没有 source 环境。

23.4 多机通信失败

检查：

```
1 echo $ROS_DOMAIN_ID
2 echo $RMW_IMPLEMENTATION
3 ip addr
4 ping other_machine_ip
```

两台机器需要：

- 网络互通；
- ROS 2 环境一致或兼容；
- ROS_DOMAIN_ID 一致；
- 防火墙不阻断 DDS 通信。

24 Linux 与 ROS 2 期末速记

24.1 Linux 命令速记

任务	命令
查看当前目录	pwd
进入目录	cd dir
查看文件	ls -l
创建目录	mkdir dir
复制文件	cp a b
复制目录	cp -r dir1 dir2
移动或重命名	mv a b
删除文件	rm file
删除目录	rm -r dir
查看文件内容	cat file

分页查看	less file
查看结尾	tail -n 20 file
实时查看日志	tail -f log
查找文本	grep "word" file
查找文件	find . -name "*.c"
查看进程	ps aux
杀死进程	kill PID
查看磁盘	df -h
查看目录大小	du -sh dir
查看网络	ip addr
远程登录	ssh user@ip
复制远程文件	scp file user@ip:/path
打包压缩	tar -czvf a.tar.gz dir
解压	tar -xzvf a.tar.gz

24.2 ROS 2 命令速记

任务	命令
查看节点	ros2 node list
查看节点信息	ros2 node info /node
查看话题	ros2 topic list
查看话题数据	ros2 topic echo /topic
查看话题频率	ros2 topic hz /topic
发布话题	ros2 topic pub /topic type data
查看服务	ros2 service list
调用服务	ros2 service call /srv type data
查看参数	ros2 param list
获取参数	ros2 param get /node param
设置参数	ros2 param set /node param value
查看接口	ros2 interface list
运行节点	ros2 run pkg exe
启动 launch	ros2 launch pkg file.launch.py
创建包	ros2 pkg create pkg --build-type ament_python
编译工作空间	colcon build
加载环境	source install/setup.bash
录包	ros2 bag record -a
回放 bag	ros2 bag play bag

25 考试常考问答

25.1 Linux 为什么说一切皆文件？

因为 Linux 将普通文件、目录、设备、管道、套接字等都抽象成文件，通过统一的文件接口进行访问。例如串口设备可以通过 `/dev/ttyS0` 访问，磁盘可以通过 `/dev/sda` 访问。

25.2 绝对路径和相对路径区别是什么？

绝对路径从根目录 `/` 开始，例如：

```
1 /home/user/test.txt
```

相对路径相对于当前目录，例如：

```
1 ./test.txt
2 ../src/main.c
```

25.3 硬链接和软链接区别是什么？

软链接类似快捷方式：

```
1 ln -s source link_name
```

硬链接：

```
1 ln source link_name
```

区别：

- 软链接有自己的 inode，指向原路径；
- 硬链接和原文件共享 inode；
- 软链接可以跨文件系统；
- 硬链接通常不能链接目录；
- 删除原文件后，软链接失效，硬链接仍可访问数据。

25.4 进程和线程区别是什么？

- 进程是资源分配的基本单位；
- 线程是 CPU 调度的基本单位；
- 一个进程可以包含多个线程；

- 同一进程内的线程共享地址空间；
- 进程之间相互独立，通信成本更高。

25.5 ROS 2 Topic、Service、Action 区别是什么？

类型	通信方式	适用场景
Topic	发布订阅	连续数据流，如图像、雷达、速度
Service	请求响应	短时间任务，如查询、开关、重置
Action	目标-反馈-结果	长时间任务，如导航、机械臂运动

25.6 ROS 2 为什么不需要 ROS Master？

ROS 2 使用 DDS 作为底层通信机制，节点之间可以自动发现彼此，不再需要 ROS 1 中的 ROS Master 进行集中式管理。

26 最后复习建议

1. Linux 命令一定要动手敲，尤其是 ls、cd、cp、mv、rm、grep、find、ps、kill、chmod、tar、ssh、scp。
2. Shell 脚本重点掌握变量、if、for、while、函数、参数、退出码。
3. 嵌入式 Linux 重点理解启动流程、设备文件、proc、sys、设备树、内核模块、交叉编译。
4. ROS 2 重点理解 Node、Topic、Service、Action、Parameter、Launch、QoS。
5. 考试遇到命令题时，先想清楚“对象是什么、操作是什么、选项是什么”。